



Portfolio sub project 1

COURSE

COMPLEX IT SYSTEMS

Group 11 members

IDA HAY JØRGENSEN	80088
JULIUS KRÜGER MADSEN	80734
MAREK LASLO	82428
SOFUS HILFLING NIELSEN	82278

October 7, 2024

Contents

1	Introduction	2
2	Application design	2
2.1	Movie Data and Framework Model	2
2.2	Entity Relationship Diagram	3
2.3	Indexing	7
2.3.1	Foreign Key Indexing	9
2.3.2	Commonly Queried Columns	9
2.3.3	Composite Indexing	10
2.3.4	Reflections on Indexing	10
3	Development	11
3.1	Rating	11
3.2	Ranking	12
3.3	Similar Movies	14
4	Reflections	16
4.1	Teamwork	16
4.2	Obstacles we encountered	16
4.3	Future improvements and bug fixes	17

1 Introduction

The application is designed to discover, track, and manage users' engagement with movies and series. The core functionalities include allowing users to search for media, bookmark titles on a personalized list, and rate media. This design is structured around a centralized media model that links various entities such as genres, cast members, and user interactions like scoring and bookmarking to media titles like movies, series and TV episodes.

2 Application design

The application is designed to allow users to search and browse through media entries using filters for genre, cast, or release year. The framework facilitates user registration and features like bookmarking and tracking watched titles, search history and ratings are also important for creating a good system. Users can bookmark any media, which will be stored in the "bookmarks" entity, allowing them to add notes. Already viewed media can also be tracked in the "completed" entity, which stores notes regarding the media and its rewatchability.

User ratings will be stored in the "user score" entity, where users can provide a score and optional review text for each media entry they have watched. The "search history" entity will track previously searched items. This modular structure ensures that users can easily manage their interactions with the media.

2.1 Movie Data and Framework Model

The application is separated into two independent models, the movie data model and the framework model. This design allows the framework model to be separated and moved to a separate database should the need arise.

The original data was structured in 7 tables plus a table with additional information about the titles and an inverted word index table used for efficient querying of the data in the database. To satisfy the Boyce–Codd normal form (BCNF) as well as accommodate new features, the structure and relations between the data in the original tables have been changed, however, most of the data has been transformed to the new tables.

Some fields in the original dataset did not adhere to BCNF. This includes attributes and types associated with title aliases, as well as genres for media entries. These non-normalized elements have now been separated into distinct tables to maintain data consistency and improve query efficiency.

In the new design, all data revolves around two entity types, **persons** and **media**, which hold all the essential information about actors and crew members as well as movies and TV shows respectively. The person entity contains attributes like name and birth date, death date and the media entity contains attributes like plot, runtime, and box office performance.

Additional information for media is represented in other entities like the **release** entity which allows for a media to have multiple release dates, this can be useful in scenarios where a movie is released at different times globally or when there are distinct physical release events. The **title** entity manages alternative titles for a single media entry. This entity is based on the data in the titles_akas table from the original dataset but has been enhanced by normalizing previously mixed data such as **title attributes** and **title types** into their respective tables.

Additionally, predefined data such as **languages**, **countries**, **job categories**, and **genres** has been moved into dedicated Data Tables. This ensures consistency across the database, particularly when linking related entities.

Finally, the system introduces custom data features not present in the original data model, including the ability to **relate media** items to one another (e.g., spin-offs or remakes) and organize media into **collections** based on common attributes.

2.2 Entity Relationship Diagram

The Entity Relationship (ER) diagram allowed us to visualize the relationships between entities, which in turn allowed us to clarify how each table interacts with each other, particularly when handling media-specific data like genres, titles, and releases. The ER diagram also played a critical role in identifying areas where normalization was needed, helping us streamline the structure of the database for better performance and maintainability.

One challenge we faced during the design of the ER diagram was how to accurately represent specialized media types, such as episodes and seasons. During our design process, we determined that episodes and seasons require extra information, unlike standalone media entries like movies. Series are represented in a hierarchical model where episodes belong to seasons, and seasons, in turn, belong to a series. We were initially unsure of how to represent this in a meaningful way since episodes and seasons are meant to be specializations for the media entity. However, after some discussions, we realized that both episodes and seasons can be thought of as a relation between two media entries which lead us to the final design (see. figure 1).

We also had an internal discussion regarding the relationship between the episode and season entities. In our design, both the parent for seasons and episodes were treated as extensions of the media table, where a parent season in the episode table referenced the media entity marked as a season. While this simplified the linking media entries, it raised concerns about potential data integrity issues.

One point raised was that this setup could allow for inconsistent foreign key references, where any media entity (even if not a season) could be linked to an episode. An alternative approach discussed was to have strict foreign key relationships between the specialized entities (episode -> season -> series). This structure would prevent invalid references and provide a clearer and more enforceable hierarchy. We concluded that both approaches have merit, and while we opted not to change the design at this stage, the discussion underscored the importance of balancing strictness with maintainability when designing a database.

When we began developing the database we realized that we needed to change how titles were structured. In our initial design (figure 1) we thought of a release as being synonymous with a title. However, we later realised that titles and releases are distinct entities and we would need to be represented in the model as so. This required us to split the title into its own entity and thereby allow both releases and titles to be stored independently from each other in the data model. This also proved beneficial in allowing multiple types and attributes for a single title.

Another enhancement to the final design was the addition of a new entity, to the framework model that allows users to follow individuals, such as actors, directors, and other industry professionals. Initially, our ER diagram for the framework model focused

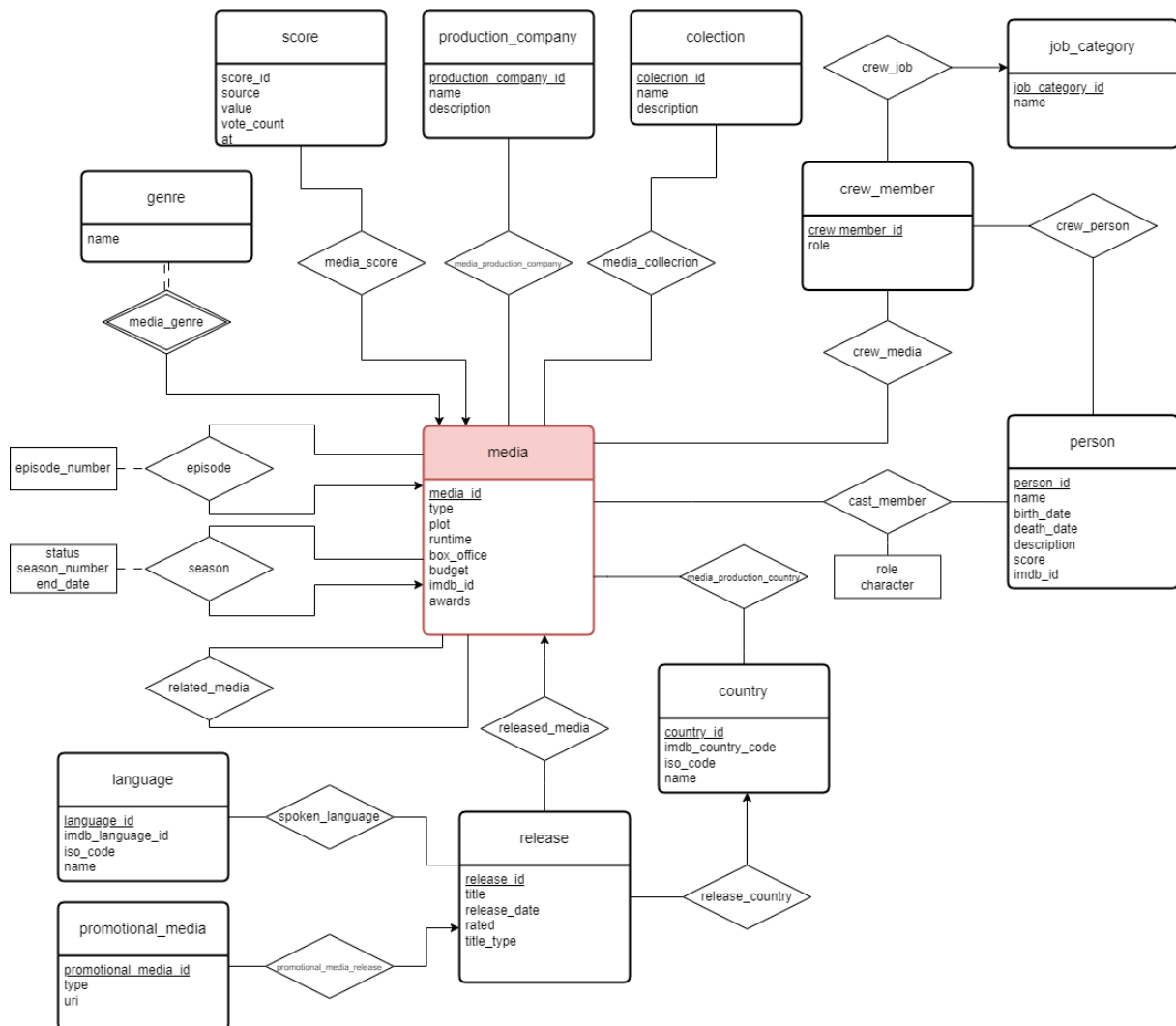


Figure 1: Handmade ER diagram for the movie data model.

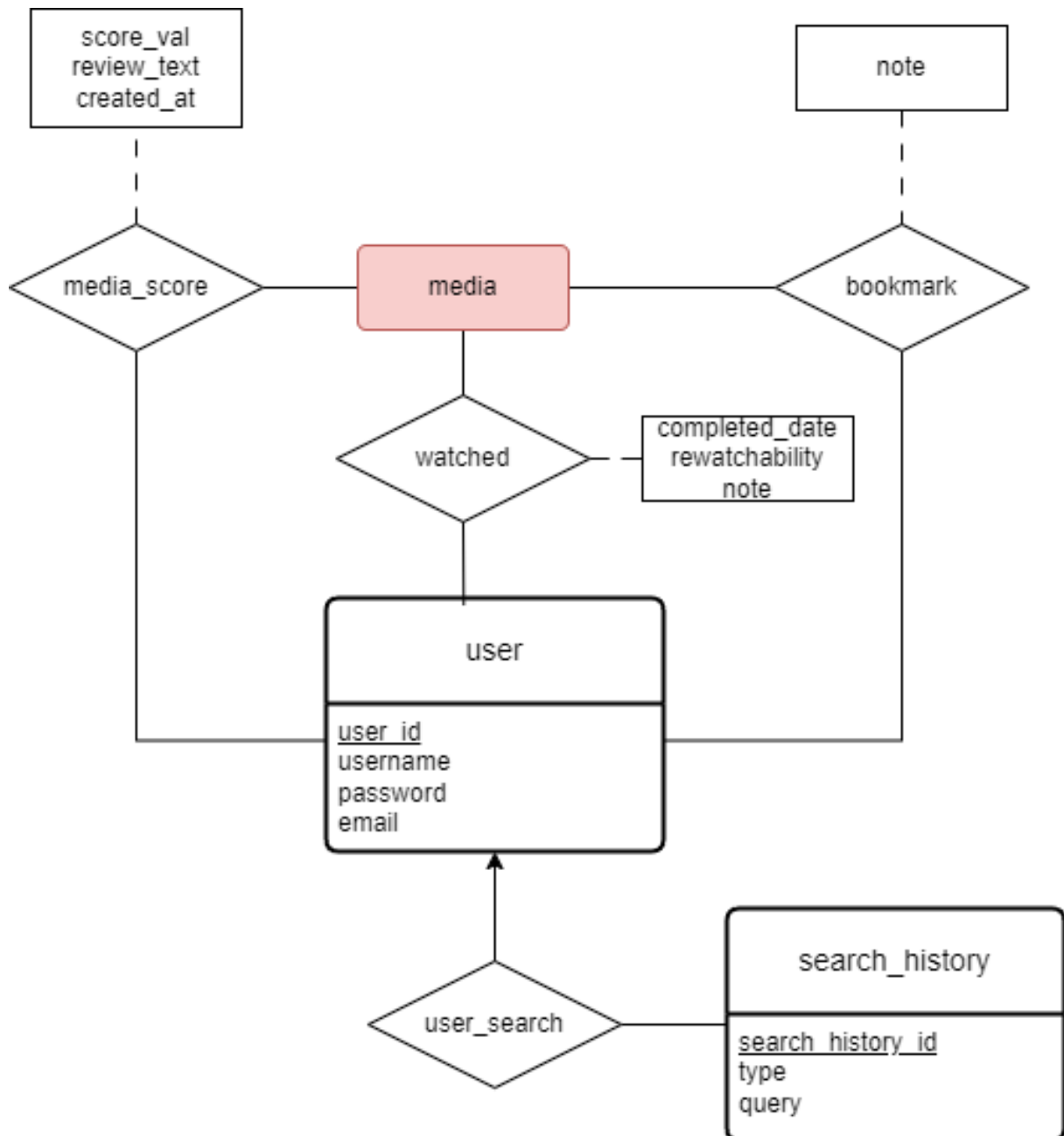


Figure 2: Handmade ER diagram for the framework data model.

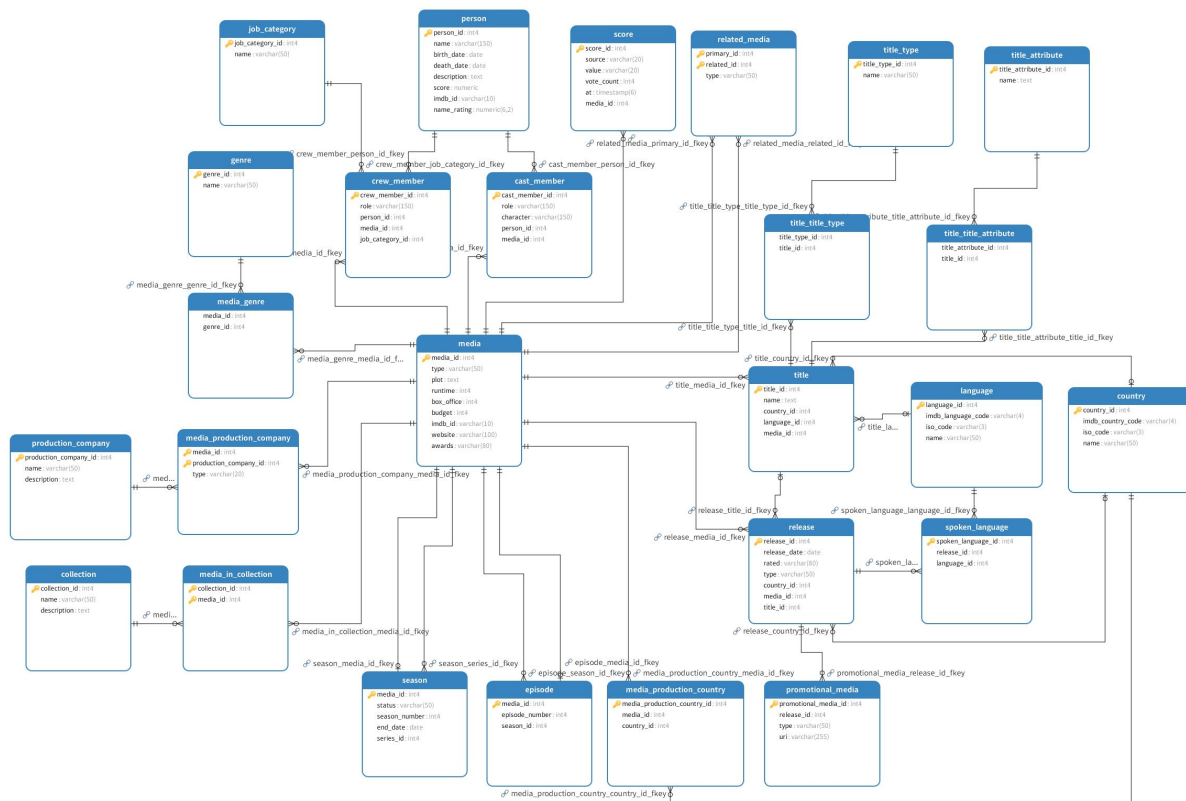


Figure 3: Reverse engineered ER diagram for the movie data model.

primarily on media entries, but we recognized the need to extend the framework to support user interactions beyond media.

By introducing an entity called **following**, shown in figure 4, we enabled users to follow specific people, keeping track of updates related to their work, such as new releases. This entity links users to individuals in the person table, which stores information about actors, directors, and other notable figures involved in media production. The relationship between users and persons is a many-to-many, as each user can follow multiple individuals, and each individual can be followed by multiple users.

2.3 Indexing

One of the advantages of using a relational database for the application's data model is the ability we have to optimize query performance through indexing. By selecting which columns to index, we can enhance our frequently executed queries, especially

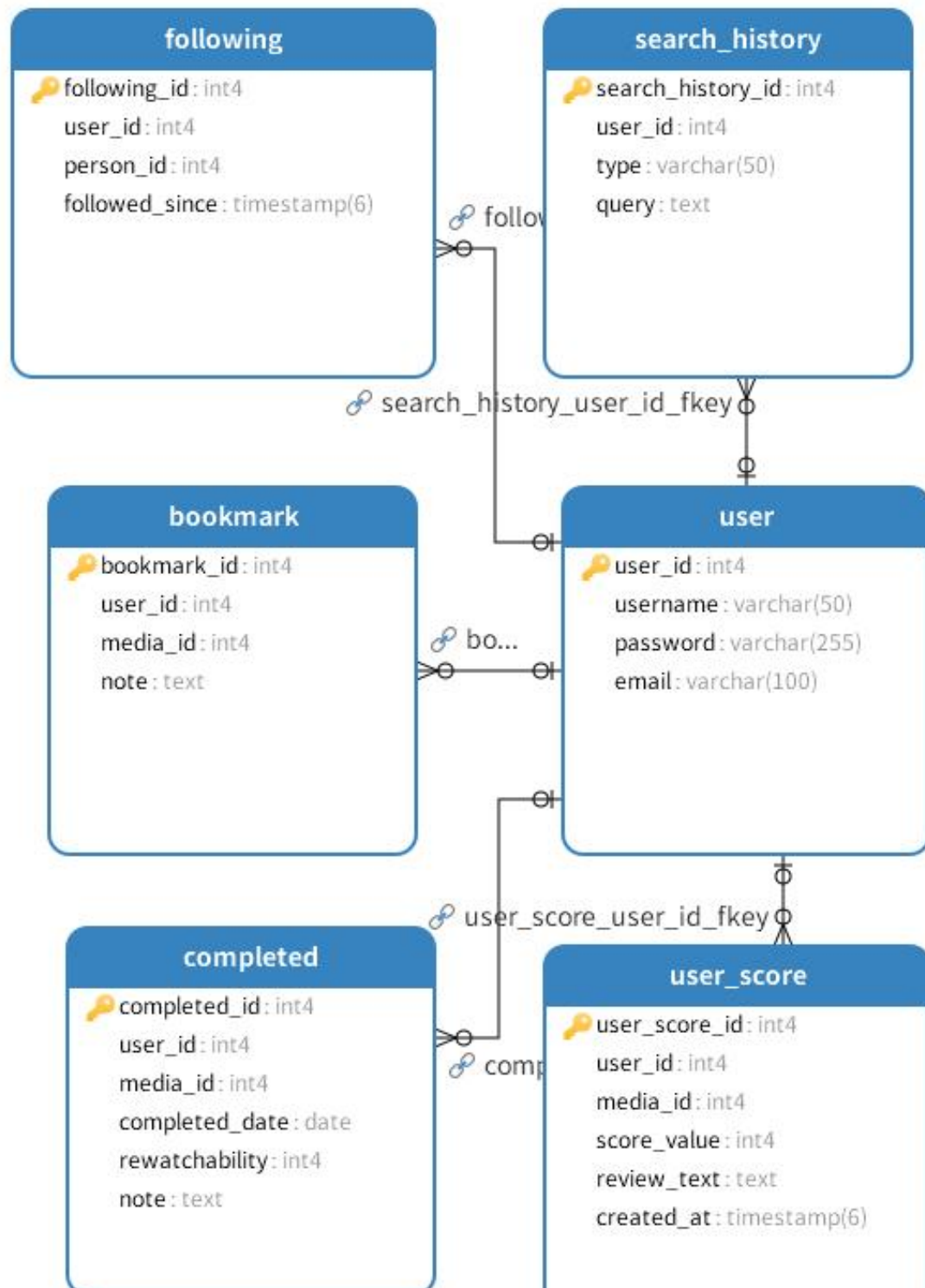


Figure 4: Reverse engineered ER diagram for the framework data model.

as the dataset grows. We also have to keep in mind that improper or excessive indexing can lead to performance drawbacks, particularly for INSERT, UPDATE and DELETE operations.

2.3.1 Foreign Key Indexing

Foreign keys are used to connect entities like users, media, and bookmarks. By indexing these foreign key columns, we can improve performance, particularly when executing joins between these related entities.

As an example the `user_id` foreign key appears in tables such as `search_history`, `bookmark`, `completed`, and `user_score`. Indexing those columns will make retrieval of user-specific data, such as bookmarks or completed media faster.

```
CREATE INDEX idx_search_history_user_id ON search_history(user_id);  
CREATE INDEX idx_bookmark_user_id ON bookmark(user_id);  
CREATE INDEX idx_completed_user_id ON completed(user_id);  
CREATE INDEX idx_user_score_user_id ON user_score(user_id);
```

We have also chosen to index the `media_id` column in multiple tables like `bookmark`, `completed`, and `user_score` to improve queries related to media lookups.

```
CREATE INDEX idx_bookmark_media_id ON bookmark(media_id);  
CREATE INDEX idx_completed_media_id ON completed(media_id);  
CREATE INDEX idx_user_score_media_id ON user_score(media_id);
```

2.3.2 Commonly Queried Columns

Certain columns are frequently queried, making them candidates for indexing. For example, the `username` and `email` columns in the `user` table are often used during login or profile lookups. Indexing these two columns will make the overall user experience better, as login will generally be faster if we have a big dataset.

```
CREATE INDEX idx_user_username ON "user"(username);  
CREATE INDEX idx_user_email ON "user"(email);
```

Also, columns like `completed_date` in the `completed` table and `score_value` in the `user_score` table are often used for sorting or filtering, which is why they are also indexed to improve performance.

```
CREATE INDEX idx_completed_completed_date ON completed(completed_date);  
CREATE INDEX idx_user_score_score_value ON user_score(score_value);
```

2.3.3 Composite Indexing

In certain cases, composite indexes are required for queries that filter based on multiple columns. A composite index on `media_id` and `country_id` in the `media_production_country` table, for example, allows for faster lookups when both the media and the country needs to be considered.

```
CREATE INDEX idx_media_production_country_media_country ON  
media_production_country(media_id, country_id);
```

2.3.4 Reflections on Indexing

While indexing can have a big impact positively on query performance, we also need to consider its drawbacks. First of all indexes use additional storage space and secondly they can slow down write-heavy operations like `INSERT`, `UPDATE`, or `DELETE`, as the indexes must also be updated whenever the data changes. In large-scale applications with frequent writes, the overhead of maintaining multiple indexes could/ most likely would impact performance.

Another consideration we needed to make was in regards to over-indexing, which occurs when too many indexes are created. This can lead to slower performance for less frequent queries and it increases the complexity of potential future database maintenance. A balanced approach to indexing is necessary to avoid this, by ensuring that

our indexes are only created for columns and queries that benefit from them.

By targeting foreign keys, commonly queried columns, and composite keys, we will improve the responsiveness and scalability of the application's database.

3 Development

This section details the development process of the required functionality as well as the challenges encountered during this process.

3.1 Rating

A rating system allows users to express their opinions about a particular media. In this case, the rating is represented as an integer value on a predefined scale between 1 and 10. A key feature of the system is the calculation of an average rating from multiple user submissions, which helps future users get an idea of how good the media is.

The function 'rate()' specifically handles the rating of media titles. It takes four parameters: the id of the user submitting the rating, the id of the media, the value of the rating (between 1 and 10), and a text-based review of the media. The function first checks if the user has already submitted a rating for the same media. If a rating exists, it updates the score and review with the new values; if not, it inserts a new rating from the user into the 'user_score' table.

This is done via the smart technique which is available in PL/pgSQL:

```
IF FOUND THEN
```

This is a smart way to handle whether a specific query has returned any results. In the function, after querying the 'user_score' table it checks if the user already has rated the media. The IF FOUND THEN checks whether the query succeeded and if it finds a row then it updates the existing entry or via the if-else logic inserts a new rating from the user.

After processing the user's submission, the function updates the average rating for the media title. It checks if the media already has an entry in the 'score' table. If an entry exists, it updates the average score and vote count. If it's the first rating for the media, a new entry is created with the calculated average score and vote count. Additionally, the function updates the timestamp, ensuring the system records when the rating was last modified, thereby creating a rating history.

Finally, the function displays the updated average rating using a `RAISE NOTICE` statement, which helps track the function's output during execution:

```
RAISE NOTICE 'Average Rating: %', (SELECT AVG(CAST(score_value AS NUMERIC))
FROM user_score
WHERE media_id = v_media_id);
```

This approach not only allows users to keep track of their ratings and reviews but also provides flexibility for users to change their ratings if they reconsider their opinions about a title.

3.2 Ranking

Ranking in search systems refers to the process of ordering or prioritizing results based on their relevance or quality in relation to a user's query. This process is crucial for delivering accurate information to users, enhancing their overall search experience.

A function that exemplifies this concept is the function to find the most frequent co-actors of a given actor (D6). This function takes a person's name as an argument and returns a table containing a ranked list of co-actors with whom the actor has collaborated. The ranking is determined by counting the number of titles they have worked on together, allowing for a clear display of frequent collaborations.

In relation to the ranking, the 'wi' table is important. This table provides an inverted title index, mapping words to their associated titles. This structure enables quick lookups for keyword-based queries, a core aspect of unstructured search, where data is not organized in a predefined schema. By utilizing this index, search functions can efficiently return relevant information, making it easier to rank results based on the

user's input. The 'wi' table is used in various functions, including the frequent-person-words-function, the exact-match-function and the word-to-words-function.

The frequent-person-words-function (D10) takes a person's name as an argument and returns the words, along with their frequencies, from the titles that the person has worked on. This is achieved by joining the 'wi', 'media', 'cast_member' and 'person' table. The query includes a condition to filter titles based on a match of the person's name using the ILIKE operator. The results are then ordered by frequency in descending order, with the most significant words related to that person appearing first. The function also accepts an optional argument that limits the number of results returned.

The exact-match-function (D11) takes a list of keywords as arguments, finds the titles that contain all the given keywords using the inverted index, and then returns the media_id and the original title of the titles that match. This is achieved using the query:

```
HAVING COUNT(DISTINCT word) = array_length(keywords, 1)
```

This condition ensures that only titles containing all the given keywords are returned. The use of the GROUP BY clause groups the results by title 'tconst', while the HAVING clause filters those results to include only titles where the number of distinct matching words equals the number of input keywords. This allows for efficient keyword matching and filtering in a single query.

An alternative approach would be to:

- Retrieving the titles for each keyword individually.
- Finding the intersection of these sets to identify the titles that appear in all keyword results.

Both methods achieve the goal of exact-match querying, however using GROUP BY and HAVING is more efficient and compact compared to executing multiple queries and manually intersecting the results.

The best-match-function (D12) takes a list of keywords as arguments and ranks titles based on the number of keywords that match. This is handled by the clause:

```
WHERE wi.word = ANY(keywords)
```

The clause ensures that only titles containing at least one of the keywords are considered. The function then counts the number of distinct matching keywords using:

```
COUNT(DISTINCT wi.word)
```

Then the titles are grouped and ordered, ensuring that those with more keyword matches are listed first.

Alternatively, there is also the possibility to match the keywords separately using `UNION ALL` and the use `SUM()` to determine the rank based on the number of keyword matches. While both approaches share the same logic, the approach using `GROUP BY` and `HAVING` is more compact and reduces complexity.

The word-to-words-function (D13) accepts a list of keywords as arguments and returns a ranked list of words based on their frequency in titles that match the given keywords. The process begins by identifying all titles containing any of the specified keywords using the `ILIKE` operator, allowing for flexible, case-insensitive pattern matching, as used in previous functions. Also `UNNEST(keywords)` are used and this transforms the array of keywords into a set of rows, making it easier to match each keyword against the title strings in the table. Once the relevant titles are identified, the function calculates the frequency of words associated with those titles. This is done by joining the 'wi' table and using the `COUNT(*)`. The function then returns the most frequent words along with their calculated frequencies in descending order based on frequency.

3.3 Similar Movies

Similar movies functionality is developed to return the list of similar media for a user specified movie or TV series. After the broad discussion of what would be the best way to develop this kind of function, we have agreed on five main notions of similarity, which can be between the media.

1. Similarity based on the title.

We developed a function that extracts all the individual words from a title and stores them in an array. The function then loops through this array, and for each

word, it retrieves a list of media IDs where the word appears as a substring in a title. After scanning and collecting the movie IDs for all the words in the selected title, the function returns the matching media IDs with the count of matching words for each.

```
get_movies_with_similar_titles(input_title TEXT, min_length INTEGER)
```

The function takes an additional parameter `min_length`, which restricts a minimum length threshold for words in a sub-string search. In our case, we set a minimum length to be three so words like: `an`, `and`, `or`, `of`, `the`, etc. are ignored.

2. Similarity based on the cast

For this notion of similarity, we retrieve a list of all the actors in the selected media and then we search for all the media, that each actor was starring in. After retrieving the list with all the other media of all the actors, we count the occurrence of each media in the list and return a list of media IDs with the occurrence count.

3. Similarity based on the production crew

Similar system as for similarity based on the cast, only this function is scanning for the movies, in which the individual production crew members were part of.

4. Similarity based on the country of origin

Similarity based on the country of origin just finds all the media from the same country as the selected media. Then it returns the list of the media with count as one for each.

5. Similarity based on the genre

The last similarity is about the genre, the logic is similar as for the logic of a cast and crew. But in this case, we find the genre names of the selected media and return all the media which match the genre. In the same way, we count the occurrence of each media and return the table with media IDs and count the number for each.

In the final function called:

```
get_similar_movies(input_media_id INTEGER)
```

We call all the similarity functions inside a final function, where each function returns a table containing media IDs and their respective similarity counts. In this main function, we aggregate the results by performing a union of all the tables, combining media IDs and summing their corresponding counts from the different similarity searches.

The final result is a list of media IDs and the total sum of the counts received from all similarity searches. This list is then sorted in descending order, where the most similar media items are those with the highest total count values.

4 Reflections

The development of the system revealed some insights related to its functionality, design, and implementation. In the following, some of the findings will be covered.

4.1 Teamwork

Throughout the assignment, we maintained an efficient workflow and handled tasks effectively. Moving forward, we plan to hold a meeting to reflect on our work on this project. During this session, we aim to identify areas for improvement and implement solutions to optimize our development process for future projects.

- Communication: Discord, Messenger
- Project planing: GitHub project board
- Git repository management tool: GitHub
- Methodology: agile

4.2 Obstacles we encountered

When designing our database, we overlooked the need to account for the redundancy of different release titles for a media item. Unfortunately, we discovered this issue late

in the process and had to make the decision to fix the database design. As a result, we were forced to rewrite most of our scripts and adjust them to fit the new structure. In the future, we aim to avoid such issues, as it led to additional work and delays in our planned delivery.

4.3 Future improvements and bug fixes

For the weighted average calculation, we did not consider that the rating data from IMDb is in the range of 0.0 to 10.0, while the values from Rotten Tomatoes and Metacritic range from 0 to 100. This oversight could cause discrepancies in the weighted average since we store all values in the same column. In the future, we aim to address and correct this issue.

After migrating data from IMDb, we found that some media release dates are incorrectly recorded as being in 1 BC. This is a minor bug that we also plan to fix in our future improvements.